Assignment 3: Implementing and Using BST

Description

In this assignment, you'll be provided an ADT for a Binary Search Tree (BSTreeADT.java) and Iterators (Iterator.java) to be implemented as a Binary Search Tree (BST). You'll then use this BST to create a tracker application, which will produce a report, specified by the user at command line, to generate output using a variety of iterators built into the BST.

Equipment and Materials

For this assignment, you will need:

- Eclipse, Java 21 and JUnit 4

Note: If your submission does not meet these requirements, you will receive a grade of zero!

Instructions

This assignment consists of three parts, to be completed **outside** of class time. See the course outline and Brightspace for due dates. Complete this assignment with your assigned group.

Part A: Create and Implement a Tracker Application using your own BST (90 marks)

1. You will implement a Binary Search Tree based on the contract in the provided ADT.
2. Using the specifications below, you'll then create an application to track error codes for using your BST implementation.

Part B: Complete an Evaluation as a Group (5 marks)

After completing your tracker application, check your work against the provided marking criteria. Your instructor will refer to your group's self-evaluation when grading the assignment and will provide further feedback and grade adjustments as needed. Your instructor is responsible for awarding the group's final grade.

1. Open the Marking Criteria document (MarkingCriteria_Assignment3.docx) and save a copy with your group's name.
2. As a group, discuss how well you met each criterion and assign yourselves a mark for each row in the table. You may include a short, point form, explanation for your mark in the Notes column.

   Note: This is an opportunity for you to identify any missing pieces according to the marking criteria, make sure you do this step diligently to avoid losing any marks that is clearly stated in this document.

© 2026, Southern Alberta Institute of Technology

3. Save this file for submission to Brightspace along with your completed code.

Part C: Complete a Peer Assessment (5 marks)

Each student must also complete a peer assessment of themselves and their group members, which will be used to determine the final calculated grades of this assignment for all group members. Your instructor will provide further submission details in class.

Assignment Specifications

Read the specifications very carefully. If you are uncertain about any of the requirements, discuss it with your instructor.

1. Import the assignment3StartingCode project into Eclipse.
2. Complete the implementation of the class **BSTree.java** using the node class **BSTreeNode.java** to each individual element of the tree. Your BST must be based on the **BSTreeADT.java** and **Iterator.java** interfaces provided by your instructor. Test your implementation using the set of JUnit tests **BSTreeTest.java.**

   Note: Do NOT modify the ADTs or tests provided in any way! Your instructor will test with the default files.

Important: *Move to the next step only when you are satisfied that the data structure above perform properly. Then implement your application as described below.*

---

3. Create a tracking application which performs the following:
   - Keep track of errors encountered from multiple sources via log files gathered.
     - For simplicity, these log files are in simple text format.
   - Read log files from the supplied at command line, one file at a time – recording all instances of errors.
     - We're only interested in the error code – which is the first string of each line in the format of "XXX-1234" where XXX is a sequence of characters and 1234 is a 4-digit value from 0000 to 9999.
   - Each unique error code will be associated with a node of the BST.
   - Since error codes can occur multiple times in a single file, and across multiple files. For each error code, record the metadata of:
     - The name of the log file

© 2026, Southern Alberta Institute of Technology

    - The line number of the error code occurrence in this log file
- The design of this metadata is up to you! What would be the “best”? 😊

4. To maintain a record across multiple runs of the program to process each log file, use serialization techniques to store the tree in a binary file called **repository.ser**.
    - Make sure you insert the class version UID to ensure backward compatibility with your repository should the class specification change with future enhancements.
5. Every time the program executes:
    - It should check whether the binary file **repository.ser** exists, and if it does, restore the tree.
    - The results of the scanning the next file should be inserted in the appropriate nodes of the tree.

Note: Therefore, **repository.ser** will contain every error code that exist in all files scanned, along with the meta information about those word locations.

6. The user should be able to run the program via the command prompt as follows:

```
java -jar Tracker.jar <input.txt> -pf/-pl/-po [-f<output.txt>]
```

    - <input.txt> is the path and filename of the text file to be processed by the Tracker program.
    - There are three mutually exclusive options at the command line:
        - **-pf** prints in in-order all error codes, along with the corresponding list of files in which the error code occur.
        - **-pl** prints in pre-order all error codes, along with the corresponding list of files and line numbers in which the error code occur.
        - **-po** prints in post-order all error codes, along with the corresponding list of files, line numbers in which the error code occur and the frequency of occurrence of the error code.
    - An optional argument to redirect the report in the previous step to the path and filename specified in **<output.txt>**.
    - Assume that the user will provide these arguments in the order shown.
7. You may use any additional Java libraries you wish to implement the tracking functionality to store the filenames and line numbers (e.g. ArrayList, HashMap).

© 2026, Southern Alberta Institute of Technology

Note: These following images are provided simply to give you an example of what the output would look like – your program output should be similar but does NOT have to match the exact formatting as long as the relevant information are shown.

Here is an example of the program with `log1.txt -pf` and no existing repo file:

```
CVD-2054 -> [log1.txt]
HGF-3387 -> [log1.txt]
KXP-7752 -> [log1.txt]
MNB-6048 -> [log1.txt]
PLK-7710 -> [log1.txt]
QTR-4821 -> [log1.txt]
RTA-9123 -> [log1.txt]
ZLD-1934 -> [log1.txt]
```

Here is an example of the program with `log1.txt -pl` and no existing repo file:

```
QTR-4821 -> [log1.txt: 1, 8]
KXP-7752 -> [log1.txt: 3]
CVD-2054 -> [log1.txt: 9]
HGF-3387 -> [log1.txt: 6]
MNB-6048 -> [log1.txt: 4]
PLK-7710 -> [log1.txt: 7]
RTA-9123 -> [log1.txt: 5]
ZLD-1934 -> [log1.txt: 2, 10]
```

Here is an example of the program with `log1.txt -po` and no existing repo file:

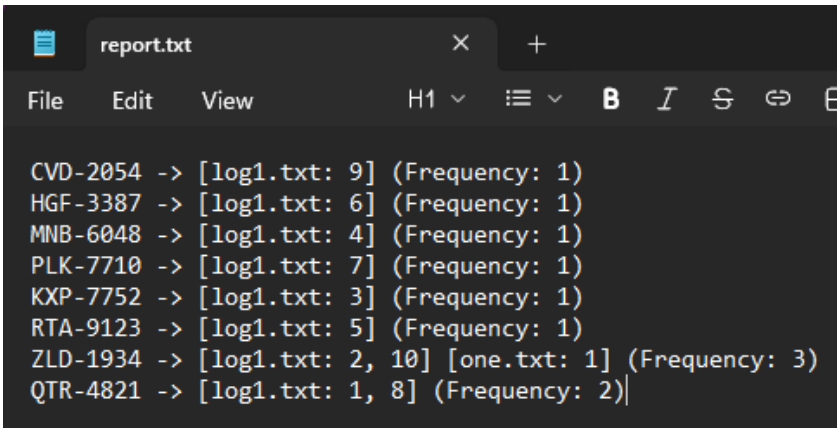
```
CVD-2054 -> [log1.txt: 9] (Frequency: 1)
HGF-3387 -> [log1.txt: 6] (Frequency: 1)
MNB-6048 -> [log1.txt: 4] (Frequency: 1)
PLK-7710 -> [log1.txt: 7] (Frequency: 1)
KXP-7752 -> [log1.txt: 3] (Frequency: 1)
RTA-9123 -> [log1.txt: 5] (Frequency: 1)
ZLD-1934 -> [log1.txt: 2, 10] (Frequency: 2)
QTR-4821 -> [log1.txt: 1, 8] (Frequency: 2)
```

Then the same program running again with a fictional file *one.txt* that has one single line with error code **ZLD-1934** and reading from the repo file from the previous run of the application:

```
CVD-2054 -> [log1.txt: 9] (Frequency: 1)
HGF-3387 -> [log1.txt: 6] (Frequency: 1)
MNB-6048 -> [log1.txt: 4] (Frequency: 1)
PLK-7710 -> [log1.txt: 7] (Frequency: 1)
KXP-7752 -> [log1.txt: 3] (Frequency: 1)
RTA-9123 -> [log1.txt: 5] (Frequency: 1)
ZLD-1934 -> [log1.txt: 2, 10] [one.txt: 1] (Frequency: 3)
QTR-4821 -> [log1.txt: 1, 8] (Frequency: 2)
```

© 2026, Southern Alberta Institute of Technology

The same program execution using the optional redirected output `-freport.txt`:



8. Make sure your code works with the other test data! (i.e., log2.txt and log3.txt)
    - Your instructor will use different test data files to evaluate your application.

Submission Deliverables

Note: This is a group assessment, so only one submission is required per group. If there are multiple submissions, your instructor will only evaluate the last submission. See Brightspace for the exact due date and time.

Your group's submission should include a zipped folder named as the assignment number and your group name (e.g., A3Zelda.zip). The zipped folder should contain the following items:

1) An **executable** Java Archive file (.jar) for your sorting application called **Tracker.jar**.
2) A zipped file of the folder containing the complete Eclipse project directory.
    a. Using File Explorer, navigate to your project folder inside your Eclipse workspace, right-click and select "Compress to… ZIP file".
3) The completed Marking Criteria document containing your group's evaluation of your application. All group members must be present when completing this document and it acts as your "sign-off" (approval) of the assignment submission.
4) Complete the Peer Evaluation Assessment as instructed by your instructor. Examples of how this evaluation will be used to determine your final grade for this assignment can be found on D2L → Unit 0: Course Information → Individual Grade Calculations for Group Assignments.pdf.

No late assignments or resubmissions will be accepted; you are responsible for the correctness of your deliverables.

© 2026, Southern Alberta Institute of Technology